

Отчёт по лабораторной работе №7

Компьютерный практикум по статистическому анализу

Надежда Александровна Рогожина

Содержание

1	Цель работы	6
2	Задание	7
3	Теоретическое введение	8
4	Выполнение лабораторной работы	9
4.1	Примеры	9
4.2	Задания для самостоятельного выполнения	21
5	Выводы	30
	Список литературы	31

Список иллюстраций

4.1	Импорт библиотек	9
4.2	Считывание данных	10
4.3	Считывание данных	10
4.4	Считывание данных	10
4.5	Запись данных в файл	11
4.6	Словари	11
4.7	Датафреймы	11
4.8	Датафреймы	12
4.9	Датафреймы	12
4.10	Датафреймы	12
4.11	Missing values	13
4.12	Missing values	13
4.13	Обработка данных	14
4.14	Обработка данных	14
4.15	Обработка данных	15
4.16	Обработка данных	15
4.17	Обработка данных	16
4.18	Обработка данных	16
4.19	Обработка данных	16
4.20	Обработка данных	17
4.21	Обработка данных	17
4.22	Кластеризация данных	18
4.23	Кластеризация данных	18
4.24	Кластеризация данных	18
4.25	Метод главных компонент	19
4.26	Метод главных компонент	19
4.27	Метод главных компонент	19
4.28	Линейная регрессия	20
4.29	Линейная регрессия	20
4.30	Линейная регрессия	21
4.31	Линейная регрессия	21
4.32	Метод k-средних для k=3	22
4.33	Метод k-средних для k=5	22
4.34	Точки и график	23
4.35	Решение в матричном виде	24
4.36	Решение <code>llsq</code>	24

4.37	Решение GLM. j 1	24
4.38	Точки	25
4.39	Функция	25
4.40	График	26
4.41	Параметры	27
4.42	Траектория курса акции	28
4.43	Функция	28
4.44	Траектория 10 акций	29
4.45	Параллельная функция	29
4.46	Траектория 10 акций	29

Список таблиц

1 Цель работы

Основной целью работы является изучение специализированных пакетов Julia для обработки данных.

2 Задание

1. Используя Jupyter Lab, повторите примеры из раздела 7.2.
2. Выполните задания для самостоятельной работы (раздел 7.4).

3 Теоретическое введение

Обработка и анализ данных, полученных в результате проведения исследований, — важная и неотъемлемая часть исследовательской деятельности. Большое значение имеет выявление определённых связей и закономерностей в имеющихся неструктурированных данных, особенно в данных больших размерностей. Выявленные в данных связи и закономерностей позволяет строить прогнозные модели с предполагаемым результатом. Для решения таких задач применяют методы из таких областей знаний как математическая статистика, программирование, искусственный интеллект, машинное обучение.

4 Выполнение лабораторной работы

4.1 Примеры

Первым делом импортируем нужные библиотеки (рис. 4.1):

```
In [1]: # Обновление окружения:
using Pkg
Pkg.update

# Установка пакетов:
using Pkg
for p in ["CSV", "DataFrames", "RODatasets", "FileIO"]
    Pkg.add(p)
end
using CSV, DataFrames, DelimitedFiles

@ Base.Docs docs\Docs.jl:243
Warning: Replacing docs for "OrdinaryDiffEqRosenbrock.ROS3PR :: Union{}" in module "OrdinaryDiffEqRosenbrock"
@ Base.Docs docs\Docs.jl:243
Warning: Replacing docs for "OrdinaryDiffEqRosenbrock.Scholz4_7 :: Union{}" in module "OrdinaryDiffEqRosenbrock"
@ Base.Docs docs\Docs.jl:243
Warning: Replacing docs for "OrdinaryDiffEqRosenbrock.ROS34PRw :: Union{}" in module "OrdinaryDiffEqRosenbrock"
@ Base.Docs docs\Docs.jl:243
Warning: Replacing docs for "OrdinaryDiffEqRosenbrock.ROS3PRL :: Union{}" in module "OrdinaryDiffEqRosenbrock"
@ Base.Docs docs\Docs.jl:243
Warning: Replacing docs for "OrdinaryDiffEqRosenbrock.ROS3PRL2 :: Union{}" in module "OrdinaryDiffEqRosenbrock"
@ Base.Docs docs\Docs.jl:243
Warning: Replacing docs for "OrdinaryDiffEqRosenbrock.ROK4a :: Union{}" in module "OrdinaryDiffEqRosenbrock"
@ Base.Docs docs\Docs.jl:243

Resolving package versions...
No Changes to 'C:\Users\Home\julia\environments\v1.11\Project.toml'
No Changes to 'C:\Users\Home\julia\environments\v1.11\Manifest.toml'
Resolving package versions...
Installed Mocketing v0.8.1
```

Рисунок 4.1: Импорт библиотек

Далее, рассмотрим примеры считывания данных (рис. 4.2, рис. 4.3, рис. 4.4), записи данных в файл (рис. 4.5), работы со словарями (рис. 4.6), работы с датафреймами (рис. 4.7, рис. 4.8, рис. 4.9, рис. 4.10), работу с переменными отсутствующего типа (рис. 4.11, рис. 4.12), работы с изображениями (рис. 4.13), обработки данных (рис. 4.14, рис. 4.15, рис. 4.16, рис. 4.17, рис. 4.18, рис. 4.19, рис. 4.20, рис. 4.21), кластеризации данных (рис. 4.22, рис. 4.23, рис. 4.24), метода главных компонент (рис. 4.25, рис. 4.26, рис. 4.27) и линейной регрессии (рис. 4.28, рис. 4.29, рис. 4.30, рис. 4.31).

Считывание данных

```
In [2]: P = CSV.File("programminglanguages.csv") |> DataFrame
Out[2]: 73x2 DataFrame                                     48 rows omitted
      Row  year  language
      ----  ---  -
      Int64 String31
1      1951  Regional Assembly Language
2      1952  Autocode
3      1954  IPL
4      1955  FLOW-MATIC
5      1957  FORTRAN
6      1957  COMTRAN
7      1958  LISP
8      1958  ALGOL 58
9      1959  FACT
10     1959  COBOL
11     1959  RPG
12     1962  APL
13     1962  Simula
```

Рисунок 4.2: Считывание данных

```
In [3]: # Функция определения по названию языка программирования года его создания:
function language_created_year(P, language::String)
    loc = findfirst(P[:,2].==language)
    return P[loc,1]
end

Out[3]: language_created_year (generic function with 1 method)

In [4]: # Пример вызова функции и определение даты создания языка Python:
language_created_year(P, "Python")

Out[4]: 1991

In [5]: # Пример вызова функции и определение даты создания языка Julia:
language_created_year(P, "Julia")

Out[5]: 2012

In [6]: language_created_year(P, "Julia")

MethodError: no method matching getindex(::DataFrame, ::Nothing, ::Int64)
The function 'getindex' exists, but no method is defined for this combination of argument types.

Closest candidates are:
  getindex(::DataFrame, ::Colon, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames C:\Users\Home\Julia\packages\DataFrames\src\dataframe\dataframe.jl:542
  getindex(::DataFrame, ::typeof(()), ::Union{Signed, Unsigned})
    @ DataFrames C:\Users\Home\Julia\packages\DataFrames\src\dataframe\dataframe.jl:548
  getindex(::DataFrame, ::InvertedIndex, ::Union{AbstractString, Signed, Symbol, Unsigned})
    @ DataFrames C:\Users\Home\Julia\packages\DataFrames\src\dataframe\dataframe.jl:538
  ...
```

Рисунок 4.3: Считывание данных

```
In [7]: # Функция определения по названию языка программирования года его создания (без учёта регистра):
function language_created_year_v2(P, language::String)
    loc = findfirst(lowercase(P[:,2]).==lowercase(language))
    return P[loc,1]
end

Out[7]: language_created_year_v2 (generic function with 1 method)

In [8]: # Пример вызова функции и определение даты создания языка Julia:
language_created_year_v2(P, "Julia")

Out[8]: 2012

In [9]: # Покстрочное считывание данных с указанием разделителя:
Tx = readin("programminglanguages.csv", ",")

Out[9]: 74x2 Matrix{Any}:
      "year"  "language"
1951  "Regional Assembly Language"
1952  "Autocode"
1954  "IPL"
1955  "FLOW-MATIC"
1957  "FORTRAN"
1957  "COMTRAN"
1958  "LISP"
1958  "ALGOL 58"
1959  "FACT"
1959  "COBOL"
1959  "RPG"
1962  "APL"
```

Рисунок 4.4: Считывание данных

Запись данных в файл

```
In [10]: # Запись данных в CSV-файл:
CSV.write("programming_languages_data2.csv", P)

Out[10]: "programming_languages_data2.csv"

In [11]: # Пример записи данных в текстовый файл с разделителем ',', ' '
writedln("programming_languages_data.txt", Tx, ',,')

In [12]: # Пример записи данных в текстовый файл с разделителем '-:', ' '
writedln("programming_languages_data2.txt", Tx, '-:')

In [13]: # Построчное считывание данных с указанием разделителя:
P_new_delim = readln("programming_languages_data2.txt", '-')

Out[13]: 74x2 Matrix{Any}:
          "year"      "language"
1951      "Regional Assembly Language"
1952      "Autocode"
1954      "IPL"
1955      "FLOW-MATIC"
1957      "FORTRAN"
1957      "COMTRAN"
1958      "LISP"
1958      "ALGOL 58"
1959      "FACT"
1959      "COBOL"
1959      "RPG"
1962      "APL"
```

Рисунок 4.5: Запись данных в файл

Словари

```
In [14]: # Инициализация словаря:
dict = Dict{Integer,Vector{String}}{()}

Out[14]: Dict{Integer, Vector{String}}{()}

In [15]: # Заполнение словаря данными:
for i = 1:size(P,1)
    year,lang = P[i,:]
    if year in keys(dict)
        dict[year] = push!(dict[year],lang)
    else
        dict[year] = [lang]
    end
end

In [18]: dict[2003]

Out[18]: 2-element Vector{String}:
         "Groovy"
         "Scala"
```

Рисунок 4.6: Словари

DataFrames

```
In [19]: df = DataFrame(year = P[:,1], language = P[:,2])

Out[19]: 73x2 DataFrame                                     48 rows omitted
          Row  year  language
          Int64  String11
1         1  1951  Regional Assembly Language
2         2  1952  Autocode
3         3  1954  IPL
4         4  1955  FLOW-MATIC
5         5  1957  FORTRAN
6         6  1957  COMTRAN
7         7  1958  LISP
8         8  1958  ALGOL 58
9         9  1959  FACT
10        10  1959  COBOL
11        11  1959  RPG
12        12  1962  APL
13        13  1962  Simula
```

Рисунок 4.7: Датафреймы

```
In [20]: # Вывод всех значения столбца year:
df[!,:year]
```

```
Out[20]: 73-element Vector{Int64}:
 1951
 1952
 1954
 1955
 1957
 1957
 1958
 1958
 1959
 1959
 1959
 1962
 1962
  ⋮
 2003
 2005
```

Рисунок 4.8: Датафреймы

```
In [23]: # Получение статистических сведений о фрейме:
describe(df)
```

```
Out[23]: 2×7 DataFrame
 Row variable mean min median max nmissing eltype
 Symbol Union... Any Union... Any Int64 DataType
 1 year 1982.09 1951 1988.0 2014 0 Int64
 2 language ALGOL 58 dBase III 0 String31
```

```
In [24]: # Подгружаем пакет Rdatasets:
using Rdatasets
```

```
In [25]: # Задаём структуру данных в виде набора данных:
iris = dataset("datasets", "iris")
```

```
Out[25]: 150×5 DataFrame 125 rows omitted
 Row Sepal.Length Sepal.Width Petal.Length Petal.Width Species
 Float64 Float64 Float64 Float64 Cat...
 1 5.1 3.5 1.4 0.2 setosa
 2 4.9 3.0 1.4 0.2 setosa
 3 4.7 3.2 1.3 0.2 setosa
 4 4.6 3.1 1.5 0.2 setosa
 5 5.0 3.6 1.4 0.2 setosa
 6 5.4 3.9 1.7 0.4 setosa
```

Рисунок 4.9: Датафреймы

```
In [26]: # Определения типа переменной:
typeof(iris)
```

```
Out[26]: DataFrame
```

```
In [27]: describe(iris)
```

```
Out[27]: 5×7 DataFrame
 Row variable mean min median max nmissing eltype
 Symbol Union... Any Union... Any Int64 DataType
 1 Sepal.Length 5.84333 4.3 5.8 7.9 0 Float64
 2 Sepal.Width 3.05733 2.0 3.0 4.4 0 Float64
 3 Petal.Length 3.758 1.0 4.35 6.9 0 Float64
 4 Petal.Width 1.19933 0.1 1.3 2.5 0 Float64
 5 Species setosa virginica 0 CategoricalValue{String, UInt8}
```

Рисунок 4.10: Датафреймы

Работа с переменными отсутствующего типа (Missing Values)

```
In [28]: # Отсутствующий тип:
a = missing
typeof(a)

Out[28]: missing

In [31]: # Пример операции с переменной отсутствующего типа:
a + 1

Out[31]: missing

In [33]: # Определение перечня продуктов:
foods = ["apple", "cucumber", "tomato", "banana"]
# Определение калорий:
calories = [missing, 47, 22, 105]

Out[33]: 4-element Vector{Union{Missing, Int64}}:
missing
 47
 22
105

In [34]: # Определение типа переменной:
typeof(calories)

Out[34]: Vector{Union{Missing, Int64}} (alias for Array{Union{Missing, Int64}, 1})
```

Рисунок 4.11: Missing values

```
In [35]: # Подключаем пакет Statistics:
using Statistics

In [36]: # Определение среднего значения:
mean(calories)

Out[36]: missing

In [37]: # Определение среднего значения без значений с отсутствующим типом:
mean(skipmissing(calories))

Out[37]: 58.0

In [40]: # Задание сведений о ценах:
prices = [0.85, 1.6, 0.8, 0.6]

# Формирование данных о калориях:
dataframe_calories = DataFrame(item=foods, calories=calories)

# Формирование данных о ценах:
dataframe_prices = DataFrame(item=foods, price=prices)

# Объединение данных о калориях и ценах:
DF = innerjoin(dataframe_calories, dataframe_prices, on=:item)

Out[40]: 4×3 DataFrame
  Row  item  calories price
  String Int64? Float64
1  apple  missing  0.85
2 cucumber    47    1.6
3  tomato    22    0.8
4  banana   105    0.6
```

Рисунок 4.12: Missing values

```
In [45]: # Загрузка изображения:
X1 = load("me.jpg")

Out[45]: 

Warning: Output switches are reduced due to the large size (640x640).
Load the ImageShow package for large images.
@ Colors C:\Users\Home\.julia\packages\Colors\VF711\src\display.jl:159

In [46]: # Определение типа и размера данных:
@show typeof(X1);
@show size(X1);

typeof(X1) = Matrix{ColorTypes.RGB{FixedPointNumbers.N0F8}}
size(X1) = (640, 640)
```

Рисунок 4.13: Обработка данных

Обработка данных: стандартные алгоритмы машинного обучения в Julia

```
In [47]: # Загрузка пакетов:
import Pkg
Pkg.add("Plots")
using Plots

Resolving package versions...
No Changes to 'C:\Users\Home\.julia\environments\v1.11\Project.toml'
No Changes to 'C:\Users\Home\.julia\environments\v1.11\Manifest.toml'

In [48]: # Загрузка данных:
houses = CSV.File("houses.csv") |> DataFrame

Out[48]: 985x12 DataFrame

          Row  street      city      zip  state  beds  baths  sq_ft  type  sale_date      price  latitude  longitude
          String String15 String15 String3 String3 String3 String15 String31 String31 String31 String31 String31
1 3526 HIGH ST      SACRAMENTO  95838  CA      2      1     836  Residential  Wed May 21 00:00:00 EDT 2008  56222  38.8319  -121.435
2  51 OMAHA CT      SACRAMENTO  95823  CA      3      1    1167  Residential  Wed May 21 00:00:00 EDT 2008  88212  38.4789  -121.431
3 2796 BRANCH ST      SACRAMENTO  95815  CA      2      1     796  Residential  Wed May 21 00:00:00 EDT 2008  88880  38.8183  -121.444
4 2805 JANETTE WAY      SACRAMENTO  95815  CA      2      1     852  Residential  Wed May 21 00:00:00 EDT 2008  86307  38.8168  -121.439
5  8001 MCMAHON DR      SACRAMENTO  95824  CA      2      1     707  Residential  Wed May 21 00:00:00 EDT 2008  81900  38.5105  -121.438
```

Рисунок 4.14: Обработка данных

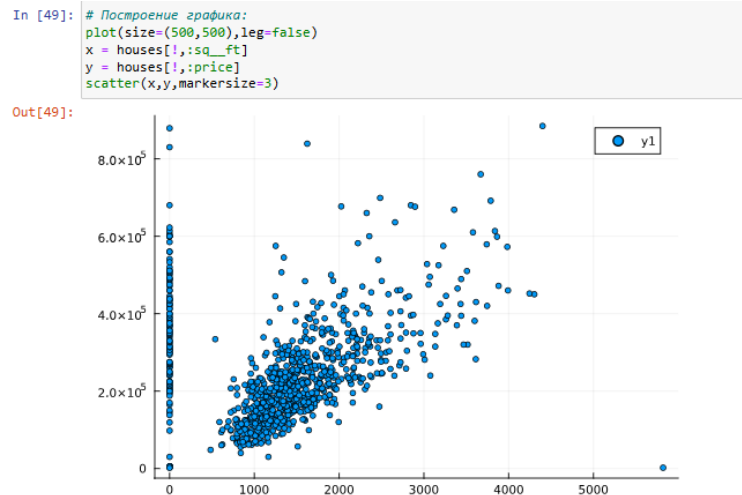


Рисунок 4.15: Обработка данных

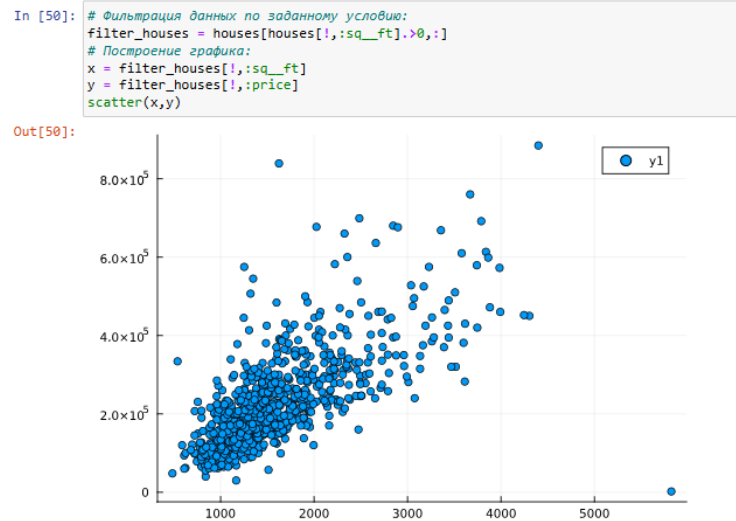


Рисунок 4.16: Обработка данных

```
In [59]: # Добавление данных :latitude и :longitude в новый фрейм:
X = filter_houses[:,[:latitude, :longitude]]
# Конвертация данных в матричный вид:
X = Matrix(X)

Out[59]: 814x2 Matrix{Float64}:
38.6319 -121.435
38.4789 -121.431
38.6183 -121.444
38.6168 -121.439
38.5195 -121.436
38.6626 -121.328
38.6817 -121.352
38.5351 -121.481
38.6212 -121.271
38.7009 -121.443
38.6377 -121.452
38.4707 -121.459
38.6187 -121.436
:
```

Рисунок 4.17: Обработка данных

```
In [60]: # Транспонирование матрицы с данными:
X = X'

Out[60]: 2x814 adjoint(::Matrix{Float64}) with eltype Float64:
38.6319 38.4789 38.6183 ... 38.7009 38.417 38.6552
-121.435 -121.431 -121.444 ... -121.257 -121.397 -121.076

In [61]: # Задание количества кластеров:
k = length(unique(filter_houses[:,zip]))

Out[61]: 66

In [62]: # Определение k-среднего:
C = kmeans(X,k)

Out[62]: KmeansResult{Matrix{Float64}, Float64, Int64}([38.6625965 38.48676566666667 ... 38.415214 38.552301999999999; -121.31647775 -121.54238366666668 ... -121.207899 -121.45057926666668], [4, 26, 39, 4, 11, 1, 55, 51, 49, 33 ... 3, 6, 26, 20, 40, 47, 26, 34, 30, 7], [4.439077019924208e-5, 0.0005054545472376049, 0.00015412842913085106, 0.00015091797467903214, 0.00015539828018518165, 0.00012848789265262894, 5.823836545459926e-5, 0.00016118244093377143, 0.0011840738843602594, 0.00011686658399412408 ... 5.784670793218538e-5, 5.2413251978578046e-5, 0.000216632652154658, 0.00023574619626742788, 0.00013415837747743353, 0.00016950622011790983, 0.0004134149457968386, 0.0006359726212394889, 0.0002555243445385713, 7.06132996128872e-5], [16, 6, 28, 21, 2, 3, 7, 4, 6, 5 ... 19, 1, 15, 6, 15, 1, 22, 20, 3, 15], [16, 6, 28, 21, 2, 3, 7, 4, 6, 5 ... 19, 1, 15, 6, 15, 1, 22, 20, 3, 15], 0.2041623775083245, 0, true)
```

Рисунок 4.18: Обработка данных

```
In [63]: # Формирование фрейма данных:
df = DataFrame(
    cluster = C.assignments,
    city = filter_houses[:,city],
    latitude = filter_houses[:,latitude],
    longitude = filter_houses[:,longitude],
    zip = filter_houses[:,zip])

Out[63]: 814x5 DataFrame 789 rows omitted
Row cluster city latitude longitude zip
Int64 String15 Float64 Float64 Int64
1 4 SACRAMENTO 38.6319 -121.435 95838
2 26 SACRAMENTO 38.4789 -121.431 95823
3 30 SACRAMENTO 38.6183 -121.444 95815
4 4 SACRAMENTO 38.6168 -121.439 95815
5 11 SACRAMENTO 38.5195 -121.436 95824
6 1 SACRAMENTO 38.6626 -121.328 95841
7 55 SACRAMENTO 38.6817 -121.352 95842
8 51 SACRAMENTO 38.5351 -121.481 95820
9 40 RANCHO CORDOVA 38.6212 -121.271 95870
10 33 RIO LINDA 38.7009 -121.443 95873
```

Рисунок 4.19: Обработка данных


```
In [64]: clusters_figure = plot(legend = false)
for i = 1:k
    clustered_houses = df[df[:,cluster].== i,:]
    xvals = clustered_houses[:,latitude]
    yvals = clustered_houses[:,longitude]
    scatter!(clusters_figure,xvals,yvals,markersize=4)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by cluster")
display(clusters_figure)
```

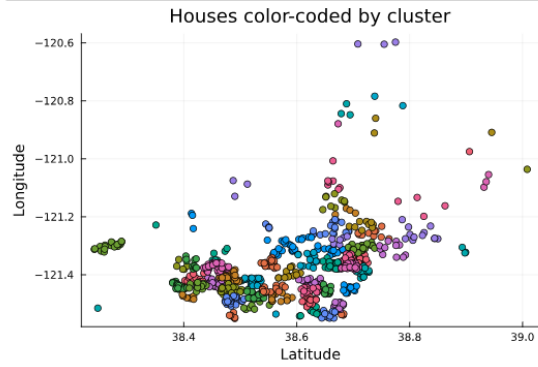


Рисунок 4.20: Обработка данных

```
In [65]: unique_zips = unique(filter_houses[:,zip])
zips_figure = plot(legend = false)
for uzip in unique_zips
    subs = filter_houses[filter_houses[:,zip].==uzip,:]
    x = subs[:,latitude]
    y = subs[:,longitude]
    scatter!(zips_figure,x,y)
end
xlabel!("Latitude")
ylabel!("Longitude")
title!("Houses color-coded by zip code")
display(zips_figure)
```

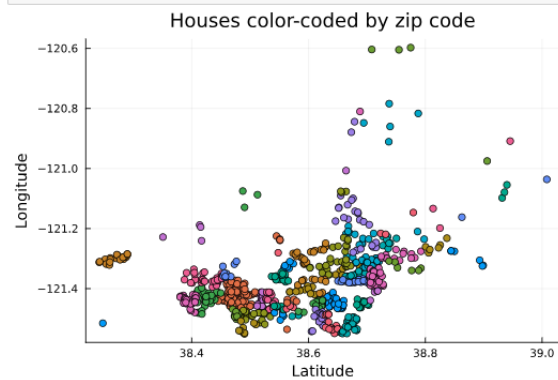


Рисунок 4.21: Обработка данных

Кластеризация данных. Метод k-ближайших соседей

```
In [66]: import Pkg
Pkg.add("NearestNeighbors")
using NearestNeighbors

Resolving package versions...
Updating "C:\Users\Home\julia\environments\v1.11\Project.toml"
[08a86587] + NearestNeighbors v0.4.24
No Changes to "C:\Users\Home\julia\environments\v1.11\Manifest.toml"

In [67]: knearest = 10
id = 70
point = X[:,id]

Out[67]: 2-element Vector{Float64}:
 38.44004
-121.421012

In [68]: # Поиск ближайших соседей:
kdtree = KDTree(X)
idxs,dists = knn(kdtree, point, knearest, true)

Out[68]: ([70, 764, 196, 125, 557, 368, 415, 92, 112, 683], [0.0, 0.006264891539364138, 0.00825320259050462, 0.008473585132630057, 0.009164073548378188, 0.009405865124697706, 0.009921759722950759, 0.009941828618812013, 0.010332637707777167, 0.011168993911721985])
```

Рисунок 4.22: Кластеризация данных

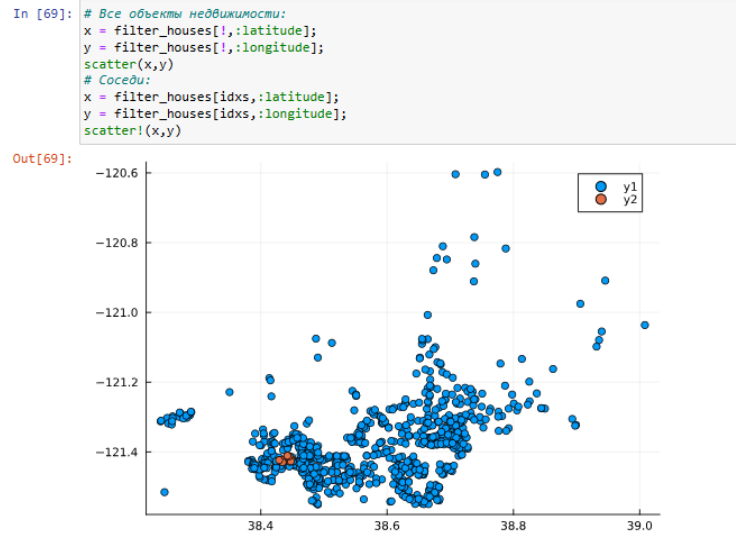


Рисунок 4.23: Кластеризация данных

```
In [70]: #Фильтрация по районам соседних домов:
cities = filter_houses[idxs, :city]

Out[70]: 10-element PooledArrays.PooledVector{String15, UInt32, Vector{UInt32}}:
 "SACRAMENTO"
 "ELK GROVE"
 "SACRAMENTO"
 "SACRAMENTO"
 "SACRAMENTO"
 "SACRAMENTO"
 "ELK GROVE"
 "ELK GROVE"
 "ELK GROVE"
 "ELK GROVE"
```

Рисунок 4.24: Кластеризация данных

Обработка данных. Метод главных компонент

```
In [88]: # Фрейм с указанием площади и цены недвижимости:
F = filter_houses[:, [:sq_ft,:price]]
y = filter_houses[:, :price]
# Конвертация данных в массив:
F = Matrix(F)'
y = y'
```

Out[88]: 1×814 adjoint(::Vector{Int64}) with eltype Int64:
59222 68212 68880 69307 81900 89921 ... 234000 235000 235301 235738

Рисунок 4.25: Метод главных компонент

```
In [89]: # Приведение типов данных к распределению для PCA:
M = fit(PCA, F)
```

Out[89]: PCA(indim = 2, outdim = 1, principalratio = 0.9999840784692097)

Pattern matrix (unstandardized loadings):

	PC1
1	460.52
2	1.19826e5

Importance of components:

	PC1
SS Loadings (Eigenvalues)	1.43584e10
Variance explained	0.999984
Cumulative variance	0.999984
Proportion explained	1.0
Cumulative proportion	1.0

Рисунок 4.26: Метод главных компонент

```
In [90]: # Выделение значений главных компонент в отдельную переменную:
Xr = reconstruct(M, y)
# Построение графика с выделением главных компонент:
scatter(F[1,:],F[2,:])
scatter!(Xr[1,:],Xr[2,:])
```

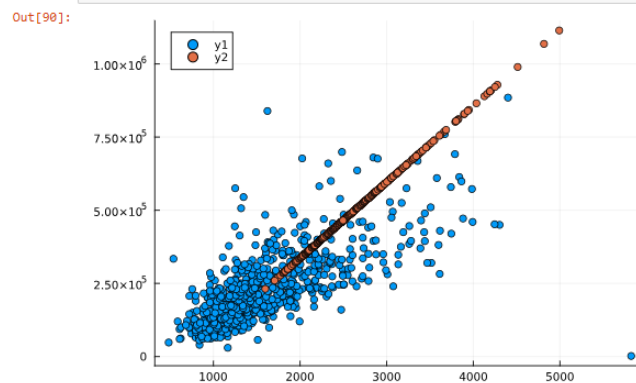


Рисунок 4.27: Метод главных компонент

Обработка данных. Линейная регрессия

```
In [91]: xvals = repeat(1:0.5:10,inner=2)
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1
scatter(xvals,yvals,color=:black,leg=false)

Out[91]:
```

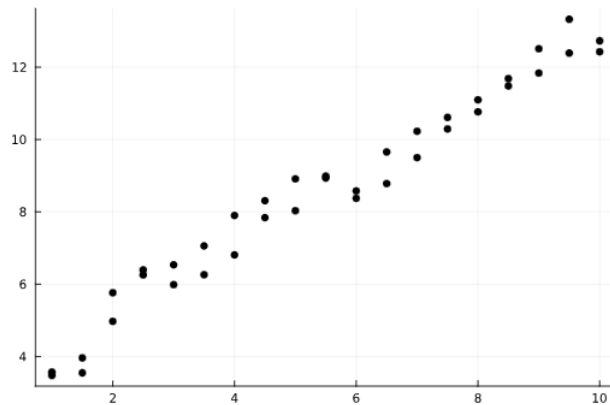


Рисунок 4.28: Линейная регрессия

```
In [92]: function find_best_fit(xvals,yvals)
meanx = mean(xvals)
meany = mean(yvals)
stdx = std(xvals)
stdy = std(yvals)
r = cor(xvals,yvals)
a = r*stdy/stdx
b = meany - a*meanx
return a,b
end

Out[92]: find_best_fit (generic function with 1 method)

In [93]: a,b = find_best_fit(xvals,yvals)
ynew = a * xvals .+ b
plot!(xvals,ynew)

Out[93]:
```

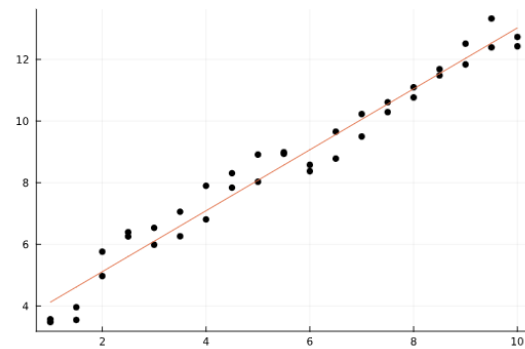


Рисунок 4.29: Линейная регрессия

```
In [98]: xvals = 1:100000;
xvals = repeat(xvals,inner=3);
yvals = 3 .+ xvals + 2*rand(length(xvals)) .- 1;
@show size(xvals)
@show size(yvals)
@time a,b = find_best_fit(xvals,yvals)

size(xvals) = (300000,)
size(yvals) = (300000,)
0.001933 seconds (5 allocations: 128 bytes)

Out[98]: (0.9999999666207922, 3.0020880560186924)
```

Рисунок 4.30: Линейная регрессия

```
In [104]: ENV["PYTHON"] = "C:/Users/Home/AppData/Local/Programs/Python/Python310/python.exe"
using PyCall

py"""
import numpy
def find_best_fit_python(xvals, yvals):
    meanx = numpy.mean(xvals)
    meany = numpy.mean(yvals)
    stdx = numpy.std(xvals)
    stdy = numpy.std(yvals)
    r = numpy.corrcoef(xvals, yvals)[0][1]
    a = r * stdy / stdx
    b = meany - a * meanx
    return a, b
"""

xpy = PyObject(xvals)
ypy = PyObject(yvals)
@time a, b = py"find_best_fit_python"(xpy, ypy)

0.161688 seconds (112.53 k allocations: 5.670 MiB, 89.54% compilation time)

Out[104]: (0.9999999666207929, 3.0020880560041405)

In [106]: import Pkg
Pkg.add("BenchmarkTools")
using BenchmarkTools
@btime a,b = py"find_best_fit_python"(xvals,yvals)
@btime a,b = find_best_fit(xvals,yvals)

Resolving package versions...
No Changes to `C:\Users\Home\.julia\environments\v1.11\Project.toml`
No Changes to `C:\Users\Home\.julia\environments\v1.11\Manifest.toml`

14.013 ms (32 allocations: 912 bytes)
758.400 μs (1 allocation: 32 bytes)

Out[106]: (0.9999999666207922, 3.0020880560186924)
```

Рисунок 4.31: Линейная регрессия

4.2 Задания для самостоятельного выполнения

1. Загрузите:

```
using RDatasets
```

```
iris = dataset("datasets", "iris")
```

Используйте Clustering.jl для кластеризации на основе k-средних. Сделайте точечную диаграмму полученных кластеров. Подсказка: вам нужно будет проиндек-

сировать фрейм данных, преобразовать его в массив и транспонировать (рис. 4.32, рис. 4.33)

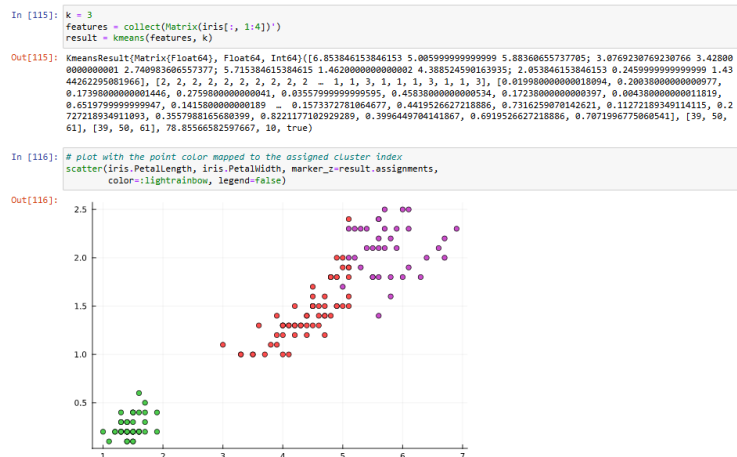


Рисунок 4.32: Метод k-средних для k=3

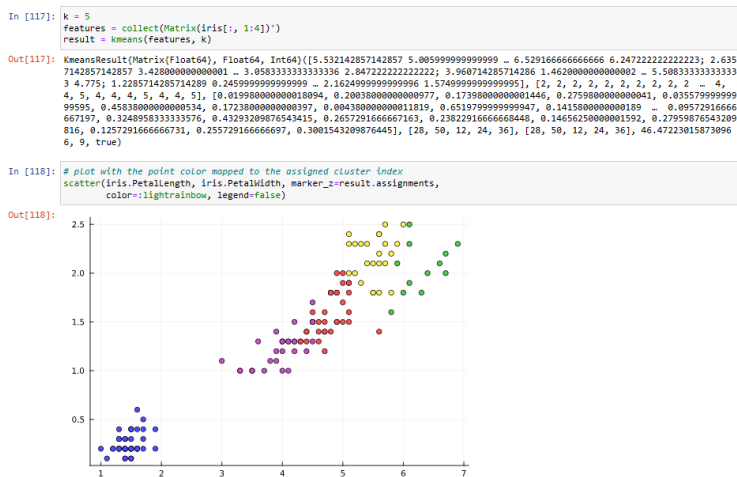


Рисунок 4.33: Метод k-средних для k=5

2. Регрессия

- Часть 1. Пусть регрессионная зависимость является линейной. Матрица наблюдений факторов X имеет размерность $N \times 3$ randn ($N, 3$), массив результатов $N \times 1$, регрессионная зависимость является линейной. Найдите МНК-оценку для линейной модели.

- Сравните свои результаты с результатами использования `llsq` из `MultivariateStats.jl` (посмотрите документацию).
- Сравните свои результаты с результатами использования регулярной регрессии наименьших квадратов из `GLM.jl`.

Подсказка. Создайте матрицу данных `X2`, которая добавляет столбец единиц в начало матрицы данных, и решите систему линейных уравнений. Объясните с помощью теоретических выкладок (рис. 4.34, рис. 4.35, рис. 4.36, рис. 4.37)

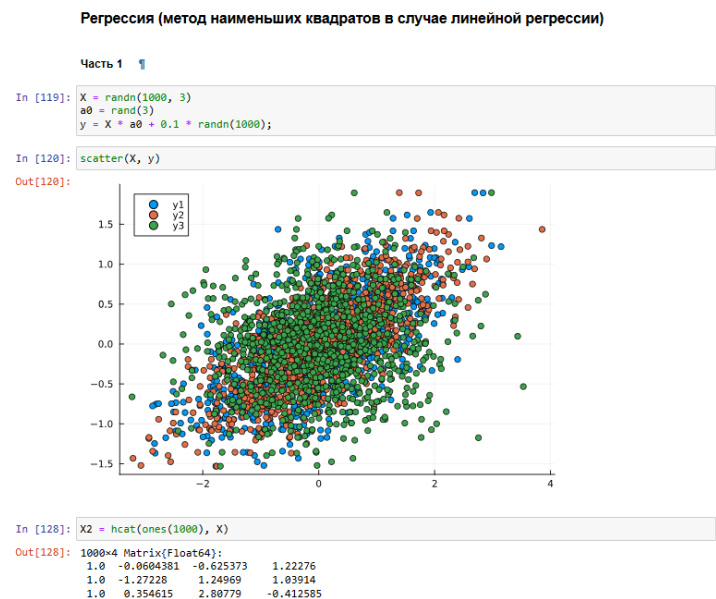


Рисунок 4.34: Точки и график

Метод наименьших квадратов (МНК) в матричной форме для определения коэффициентов регрессионных ММ

$$A = (X'X)^{-1}X'Y$$

где $X = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \dots & \dots \\ 1 & x_n \end{pmatrix}$ матрица независимых факторов x , в первый столбец этой матрицы обязательно записываются только 1. Состав остальных столбцов зависит от предполагаемой математической формулы модели.

$Y = \begin{pmatrix} y_1 \\ y_2 \\ \dots \\ y_n \end{pmatrix}$ матрица результирующих значений процесса или работы производственной системы

```
In [135]: my_beta = inv(X2'X2)*(X2'y)
Out[135]: 4-element Vector{Float64}:
 0.0011203350249490004
 0.3610800179894922
 0.42491389485302833
 0.05337234966985139
```

Рисунок 4.35: Решение в матричном виде

```
In [136]: multi_beta = llsq(X, y)
```

```
Out[136]: 4-element Vector{Float64}:
 0.3610800179894922
 0.4249138948530283
 0.0533723496698514
 0.0011203350249489991
```

```
In [138]: import Pkg
Pkg.add("GLM")
using DataFrames, GLM
```

Рисунок 4.36: Решение llsq

```
In [145]: data = DataFrame(y=y, x1=X[:,1], x2=X[:,2], x3=X[:,3])
model = lm(@formula(y ~ x1 + x2 + x3), data)
Out[145]: StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Vector{Float64}}, GLM.DensePredChol{Float64, LinearAlgebra.CholeskyPivoted{Float64, Matrix{Float64}}, Vector{Int64}}}}, Matrix{Float64}}

y ~ 1 + x1 + x2 + x3

Coefficients:

```

	Coef.	Std. Error	t	Pr(> t)	Lower 95%	Upper 95%
(Intercept)	0.00112034	0.00313985	0.36	0.7213	-0.00504114	0.00728181
x1	0.36108	0.00308864	116.91	<1e-99	0.355019	0.367141
x2	0.424914	0.0031131	136.49	<1e-99	0.418805	0.431023
x3	0.0533723	0.00314545	16.97	<1e-56	0.0471999	0.0595448

```
In [148]: coef(model)
Out[148]: 4-element Vector{Float64}:
 0.0011203350249489982
 0.3610800179894922
 0.4249138948530283
 0.0533723496698514
```

Рисунок 4.37: Решение GLM. j 1

- Часть 2. Найдите линию регрессии, используя данные (X, y) . Постройте точечный график. Добавьте линию регрессии, используя `abline`!. Добавьте заголовок «График регрессии» и подпишите оси x и y (рис. 4.38, рис. 4.39, рис. 4.40).

Часть 2

```
In [150]: X = rand(100);
          y = 2X + 0.1 * randn(100);
          xvals = X
          yvals = y

Out[150]: 100-element Vector{Float64}:
           1.773324376451762
           0.630648690935898
           1.9038255143437033
           0.8297848222663882
           1.7041890995285516
```

Рисунок 4.38: Точки

```
In [151]: function find_best_fit(xvals,yvals)
           meanx = mean(xvals)
           meany = mean(yvals)
           stdx = std(xvals)
           stdy = std(yvals)
           r = cor(xvals,yvals)
           a = r*stdy/stdx
           b = meany - a*meanx
           return a,b
       end

Out[151]: find_best_fit (generic function with 1 method)

In [152]: @time a,b = find_best_fit(xvals,yvals)

           0.019767 seconds (4.20 k allocations: 248.469 KiB, 99.31% compilation time)

Out[152]: (2.030013552566279, -0.006339733208682419)
```

Рисунок 4.39: Функция

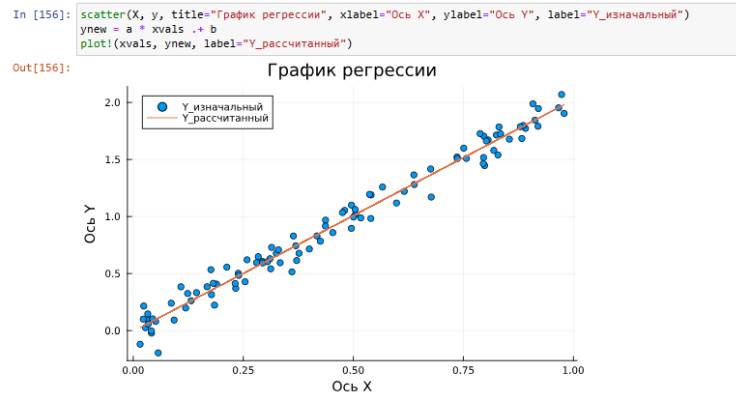


Рисунок 4.40: График

3. Модель ценообразования биномиальных опционов:

Описание модели ценообразования биномиальных опционов можно найти на [стр.https://en.wikipedia.org/wiki/Binomial_options_pricing_model](https://en.wikipedia.org/wiki/Binomial_options_pricing_model).

Постройте траекторию возможных цен на акции:

- S — начальная цена акции;
- T — длина биномиального дерева в годах;
- n — количество периодов;
- $h = Tn$ — длина одного периода;
- σ — волатильность акции;
- r — годовая процентная ставка;
- $u = \exp(rh + \sigma\sqrt{h})$;
- $d = \exp(rh - \sigma\sqrt{h})$;
- $p^* = \frac{\exp(rh) - d}{u - d}$.

а) Пусть $S = 100$, $T = 1$, $n = 10000$, $\sigma = 0.3$ и $r = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков.

- b) Создайте функцию `createPath (S :: Float64, r :: Float64, sigma :: Float64, T :: Float64, n :: Int64)`, которая создает траекторию цены акции с учетом начальных параметров. Используйте `createPath`, чтобы создать 10 разных траекторий и построить их все на одном графике.
- c) Распараллельте генерацию траектории. Можете использовать `Threads.@threads`, `map` и **`parallel`**.
- d) Пусть $S = 100$, $T = 1$, $n = 10000$, $r = 0.3$ и $\sigma = 0.08$. Попробуйте построить траекторию курса акций. Функция `rand ()` генерирует случайное число от 0 до 1. Вы можете использовать функцию построения графика из библиотеки графиков (рис. 4.40, рис. 4.41, рис. 4.42, рис. 4.43, рис. 4.44, рис. 4.45, рис. 4.46).

Модель ценообразования биномиальных опционов

```

In [178]: S = 100
          T = 1
          n = 10000
          sigma = 0.3
          r = 0.08
          h = T/n
          u = exp(r*h + sigma*sqrt(h))
          d = exp(r*h - sigma*sqrt(h))
          p = (exp(r*h)-d)/(u-d)

Out[178]: 0.49925000056249286

In [179]: prices = []
          price = S
          push!(prices, price)

Out[179]: 1-element Vector{Any}:
           100

```

Рисунок 4.41: Параметры

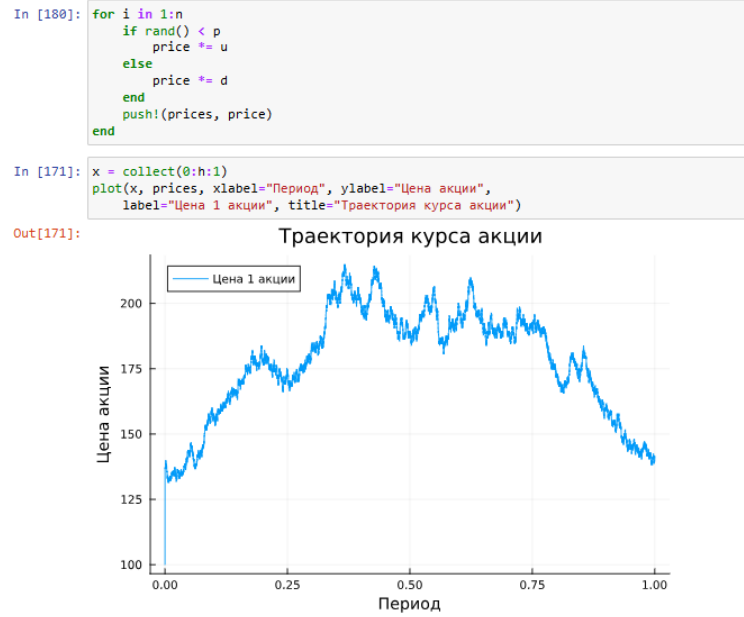


Рисунок 4.42: Траектория курса акции

```
In [181]: function createPath(S::Float64, r::Float64, sigma::Float64, T::Float64, n::Float64)
           h = T/n
           u = exp(r*h + sigma*sqrt(h))
           d = exp(r*h - sigma*sqrt(h))
           p = (exp(r*h)-d)/(u-d)
           prices = []
           price = S
           push!(prices, price)
           for i in 1:n
             if rand() < p
               price *= u
             else
               price *= d
             end
             push!(prices, price)
           end
           x = collect(0:h:1)
           return(x, prices)
         end
```

Out[181]: createPath (generic function with 1 method)

Рисунок 4.43: Функция

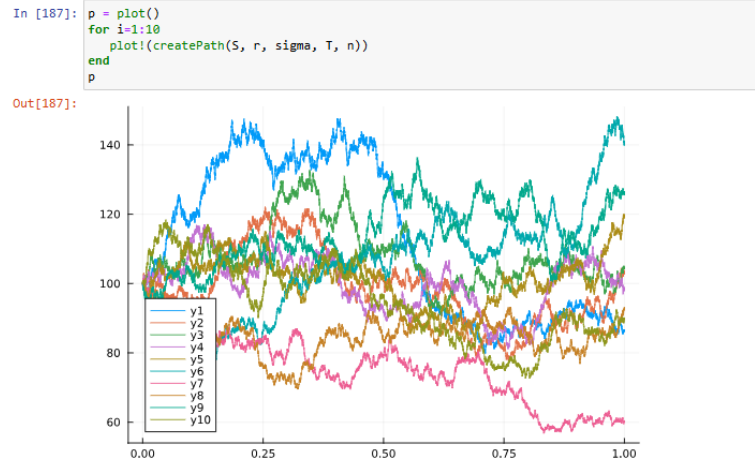


Рисунок 4.44: Траектория 10 акций

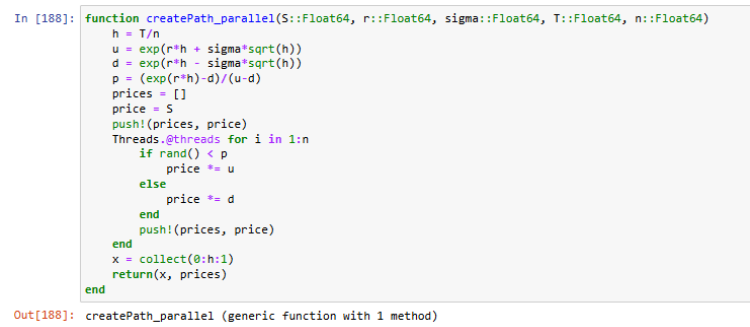


Рисунок 4.45: Параллельная функция

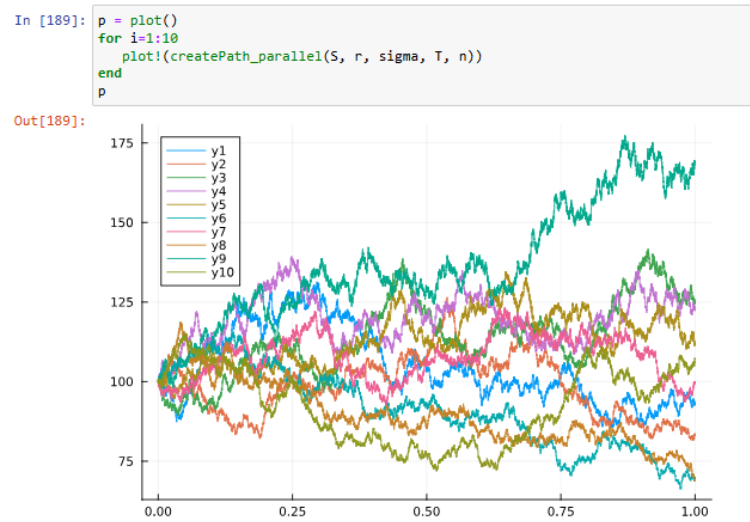


Рисунок 4.46: Траектория 10 акций

5 Выводы

В ходе работы были изучены возможности Julia в использовании специализированных пакетов Julia для обработки данных.

Список литературы